



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

LangGraph — StateGraph, nodes, conditional edges e HITL

Aula 13/14 — Módulo 4 · Grafos de estado em código

Prof. Jorge Luiz Gomes

1h40min

StateGraph · MemorySaver · HITL · draw_mermaid

Andaime 60%

Agenda da Aula

- 1 Do diagrama da Aula 12 ao código: TypedDict, add_node, add_edge, compile 10 min
- 2 add_conditional_edges — o ponto onde o grafo decide o próximo passo 12 min
- 3 draw_mermaid() — visualizar o grafo, incluindo loops 8 min
- 4 MemorySaver + checkpointing — thread_id, get_state, estado persistente 12 min
- 5 HITL com interrupt_before — pausar para aprovacao humana antes de acao irreversivel 13 min
- 6 Demo ao vivo — agente pesquisador com loop condicional (buscar→avaliar→decidir) 15 min
- 7 Lab — grafo pesquisador para o dominio do grupo (60% lacunas) 30 min

OBJETIVO DA AULA

Implementar em código o modelo mental de grafo da Aula 12. Entender como StateGraph, conditional edges, MemorySaver e HITL se encaixam. Construir um agente pesquisador com loop de avaliação de qualidade e visualizar o grafo com draw_mermaid().

Glossário

StateGraph

Classe do LangGraph que representa o grafo como objeto: recebe um schema de estado (TypedDict), agrega nodes com `add_node()` e edges com `add_edge()/add_conditional_edges()`, e compila em um grafo executável.

TypedDict (estado do grafo)

Estrutura tipada que define os campos do estado compartilhado entre nodes — cada node lê parte dela e retorna um dicionário parcial de atualização.

add_conditional_edges()

Método que registra uma aresta cujo próximo node é escolhido em tempo de execução por uma função de roteamento, permitindo loops e desvios condicionais no grafo.

draw_mermaid()

Método que gera automaticamente o diagrama Mermaid do grafo compilado — código e documentação visual nunca dessincronizam, pois vêm da mesma fonte.

MemorySaver

Checkpointing do LangGraph que persiste o estado do grafo por `thread_id`, permitindo retomar uma execução exatamente de onde parou.

thread_id

Identificador usado pelo MemorySaver para isolar o histórico de estado de execuções distintas do mesmo grafo — equivalente ao `session_id` de uma conversa.

interrupt_before

Parâmetro de compilação que pausa a execução do grafo antes de um node específico, esperando aprovação humana antes de prosseguir — mecanismo central do HITL.

HITL (Human-in-the-loop)

Padrão onde a execução automática é interrompida em pontos críticos para que uma pessoa revise ou aprove antes de uma ação irreversível continuar.

01

Do diagrama ao código

Cada elemento do diagrama mental da Aula 12 tem um equivalente direto na API LangGraph. Estado, nodes, arestas fixas, arestas condicionais, START e END.

Mapeamento: diagrama → API LangGraph

ELEMENTO DO DIAGRAMA (A12)	EQUIVALENTE LANGGRAPH	O QUE FAZ
Estado (caixas de dados)	<code>TypedDict com Annotated[list, add_messages]</code>	Dict compartilhado. Qualquer node pode ler e escrever. O grafo faz merge automático dos updates.
Nodes (retângulos)	<code>graph.add_node("nome", função)</code>	Registra uma função Python como no do grafo. A função recebe o estado e retorna um dict com as mudanças.
Arestas fixas (setas simples)	<code>graph.add_edge("no_A", "no_B")</code>	No A sempre vai para No B. Equivalente ao fluxo linear das chains LCEL.
Arestas condicionais (ramificada)	<code>graph.add_conditional_edges("no", fn, {"a": "no_A", "b": "no_B"})</code>	Chama fn(state) para decidir o próximo no. O dict traduz o retorno da função para o nome do próximo no.
START / END	<code>from langgraph.graph import START, END</code>	Nos especiais: START injeta o input no estado; END finaliza e retorna o estado final.
Compilar o grafo	<code>app = graph.compile()</code>	Valida a estrutura e retorna um Runnable compatível com .invoke(), .stream(), .ainvoke().

TUDO QUE VOCÊ JÁ SABE CONTINUA FUNCIONANDO

O grafo compilado é um Runnable LangChain. Você pode chamar com `app.invoke()`, fazer streaming com `app.stream()` e conectar ao Gradio da Aula 08. LangGraph não substitui nada — adiciona estrutura ao que já existe.

02

StateGraph em código

TypedDict define o contrato do estado. `add_node()` e `add_edge()` constroem o grafo. `compile()` valida e retorna um Runnable.

StateGraph — estrutura minima funcional

STATEGRAPH_BÁSICO.PY

```
!pip install langgraph langchain-ollama -q

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from typing import TypedDict, Annotated
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")
llm = ChatOllama(model="qwen3.6:27b", temperature=0)

# 1. Definir o Estado com TypedDict
class Estado(TypedDict):
    # Annotated[list, add_messages] usa reducer que ACUMULA (não substitui)
    mensagens: Annotated[list, add_messages]
    iteracoes: int # campo simples - substituido a cada update
    rota: str # campo simples - a decisão do classificador

# 2. Definir Nodes - funções que recebem state e retornam dict
def node_responder(state: Estado) -> dict:
    resposta = llm.invoke(state["mensagens"])
    return {
        "mensagens": [resposta], # add_messages ACUMULA - não substitui
        "iteracoes": state["iteracoes"] + 1,
    }

# 3. Construir o Grafo
builder = StateGraph(Estado)
builder.add_node("responder", node_responder)
builder.add_edge(START, "responder")
builder.add_edge("responder", END)

# 4. Compilar - valida e retorna Runnable
app = builder.compile()

# 5. Invocar como qualquer chain LCEL
resultado = app.invoke({"mensagens": [HumanMessage("Oi!")], "iteracoes": 0, "rota": ""})
print(resultado["mensagens"][-1].content)
```

03

add_conditional_edges()

O ponto onde o grafo decide o próximo passo. Uma função lê o estado e retorna uma string; o dict de mapeamento traduz para o nome do próximo nó.

add_conditional_edges() — o superpoder do LangGraph

O Router da Aula 12 em código Python. Com LangGraph, a condicao fica **explicita no grafo** — visível, testável e visualizável com `draw_mermaid()`:

CONDITIONAL_EDGES.PY

```
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel
from typing import TypedDict, Annotated, Literal
from langgraph.graph.message import add_messages

class Estado(TypedDict):
    mensagens: Annotated[list, add_messages]
    rota: str
    iteracoes: int

class Rota(BaseModel):
    destino: Literal["rag", "calculadora", "conversa"]

# Node: classifica a intencao e escreve "rota" no estado
def node_classificar(state: Estado) -> dict:
    ultima_msg = state["mensagens"][-1].content
    rota_obj = chain_clf.invoke({"input": ultima_msg})
    return {"rota": rota_obj.destino}

# Funcao de roteamento - le o estado e retorna string
def decidir_rota(state: Estado) -> str:
    return state["rota"] # deve retornar uma das chaves do mapa abaixo

builder = StateGraph(Estado)
builder.add_node("classificar", node_classificar)
builder.add_node("rag", node_rag)
builder.add_node("calculadora", node_calc)
builder.add_node("conversa", node_chat)
builder.add_edge(START, "classificar")

# add_conditional_edges(no_origem, fn_rota, {retorno_fn: nome_no_destino})
builder.add_conditional_edges(
    "classificar",
    decidir_rota,
    {"rag": "rag", "calculadora": "calculadora", "conversa": "conversa"},
)
for no in ["rag", "calculadora", "conversa"]:
    builder.add_edge(no, END)
app = builder.compile()
```

draw_mermaid() — o grafo se auto-documenta

Um dos maiores benefícios do LangGraph: o grafo pode se auto-documentar em formato Mermaid — o mesmo que o GitHub renderiza em Markdown:

```
from IPython.display import Image, display

# Renderizar como imagem PNG
try:
    display(Image(app.get_graph().draw_mermaid_png()))
except Exception:
    # Fallback: imprimir código Mermaid (colar em mermaid.live)
    print(app.get_graph().draw_mermaid())

# Saida do draw_mermaid() para o Router desta aula:
# graph TD
#   __start__([start]) --> classificar
#   classificar --> rag
#   classificar --> calculadora
#   classificar --> conversa
#   rag --> __end__([end])
#   calculadora --> __end__([end])
#   conversa --> __end__([end])
# Cole em https://mermaid.live para visualizar

# Inspeccionar nos executados com stream()
for evento in app.stream({"mensagens": [HumanMessage("Qual o prazo?")],
                          "rota": "", "iteracoes": 0}):
    print(f"No executado: {list(evento.keys())}")
# → No executado: ['classificar']
# → No executado: ['rag']
```

POR QUE DRAW_MERMAID() IMPORTA ALÉM DA VISUALIZACAO

Em sistemas em producao, a capacidade de auto-documentar o fluxo de execução é essencial para auditoria. Um grafo LangGraph pode ser exportado como Mermaid, incluído no README e renderizado automaticamente pelo GitHub — o diagrama fica sempre sincronizado com o código.

04

MemorySaver e checkpointing

Salvar e retomar o estado entre chamadas. Thread IDs permitem conversas paralelas sem interferencia. Essencial para HITL.

MemorySaver — estado persistente por thread_id

Sem MemorySaver, cada `app.invoke()` começa do zero. Com MemorySaver, o estado é salvo por **thread_id** e recuperado na próxima chamada:

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()

# Compilar COM checkpointer – habilita persistencia e HITL
app = builder.compile(checkpointer=checkpointer)

# thread_id identifica a "sessao" – cada usuario tem seu proprio thread
config_u1 = {"configurable": {"thread_id": "usuario_01"}}
config_u2 = {"configurable": {"thread_id": "usuario_02"}}

# Chamada 1 – usuario 01
app.invoke(
    {"mensagens": [HumanMessage("Qual o prazo de garantia?")],
     "rota": "", "iteracoes": 0},
    config=config_u1,
)

# Chamada 2 – mesmo usuario 01 – histórico preservado
r = app.invoke(
    {"mensagens": [HumanMessage("E a multa por rescisao?")]},
    config=config_u1,
)
print(len(r["mensagens"])) # → 4 (2 Human + 2 AI acumulados)

# Inspeccionar o estado salvo
snapshot = app.get_state(config_u1)
print(f"Iteracoes: {snapshot.values['iteracoes']}")
print(f"Proximo no: {snapshot.next}") # [] se terminou, ['nome'] se pausado
```

MEMORYSAVER VS CONVERSATIONBUFFERMEMORY (AULA 02)

ConversationBufferMemory guarda apenas mensagens para 1 conversa por vez. MemorySaver guarda todo o estado do grafo (incluindo campos customizados) e suporta multiplas sessoes paralelas por thread_id. Para producao real, usar SqliteSaver ou PostgresSaver (a API é identica, só troca o checkpointer).

05

Human-in-the-Loop (HITL)

`interrupt_before` pausa o grafo antes de um nó específico. O estado é salvo. O humano aprova (ou modifica) e o grafo continua de onde parou — impossível com `AgentExecutor`.

HITL com interrupt_before — aprovacao antes de acao irreversivel

HITL.PY

```
from langgraph.checkpoint.memory import MemorySaver

# Compilar COM interrupt_before – pausa ANTES do no especificado
app_hitl = builder.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["enviar_email"], # pausa ANTES deste no
)

config = {"configurable": {"thread_id": "tarefa_01"}}

# FASE 1 – rodar até a pausa
app_hitl.invoke(
    {"mensagens": [HumanMessage("Gere um rascunho de email")]},
    config=config,
)

# → nos anteriores executaram, PAUSOU antes de "enviar_email"
# → estado salvo no MemorySaver – thread "suspensa"

# FASE 2 – humano le o rascunho
snapshot = app_hitl.get_state(config)
print(f"Rascunho: {snapshot.values['mensagens'][-1].content}")
print(f"No suspenso: {snapshot.next}") # → ('enviar_email',)

# FASE 3a – humano aprova → continuar (input=None)
aprovado = input("Aprovar envio? (s/n): ")
if aprovado.lower() == "s":
    app_hitl.invoke(None, config=config) # None = retomar sem novo input
    print("Email enviado!")
# FASE 3b – humano edita → update_state() antes de retomar
else:
    nova_msg = HumanMessage(input("Editar rascunho: "))
    app_hitl.update_state(config, {"mensagens": [nova_msg]})
    app_hitl.invoke(None, config=config)
```

POR QUE HITL E ESSENCIAL EM PRODUCAO

Acoes irreversiveis (enviar email, debitar conta, publicar post, deletar registro) nunca devem ser executadas autonomamente sem revisao humana. interrupt_before garante que o humano sempre aprova antes de agir — independentemente do que o LLM decidiu. Essa e a diferenca entre um sistema confiavel e um risco operacional.

FUNDAMENTAÇÃO — HITL COMO FASE DE CONFIANÇA, NÃO TRAVA PERMANENTE

Bornet e Wirtz (*Agentic Artificial Intelligence*, World Scientific, 2025) descrevem human-in-the-loop com uma analogia direta: trate o agente como um funcionário recém-contratado — ninguém deixa uma contratação recente operar sem supervisão, e o mesmo vale para um agente de IA. Os autores enquadram HITL como algo especialmente importante nas fases iniciais de adoção, não como trava permanente: a colaboração funciona quando o humano mantém controle sobre metas e revisa decisões críticas, sobretudo em áreas de risco alto como financeiro ou saúde. Essa é uma camada que complementa o critério desta aula: além de "essa ação pode ser desfeita?", vale perguntar "quanto esse agente já provou ser confiável nessa tarefa?" — o mesmo nó pode justificar `interrupt_before` hoje e dispensá-lo depois de meses de execução sem erro.

HITL — o fluxo temporal com pausa

FASE 1 **invoke(input)** — grafo roda até o `interrupt_before=["enviar_email"]`. Execucao SUSPENSA. Estado salvo.

↓ estado salvo

FASE 2 **get_state(config)** — humano le o rascunho. `snapshot.next = ('enviar_email',)`

↓ humano decide

APROVADO

`invoke(None, config)` → retoma do no
suspense → email enviado → END

EDITAR

`update_state()` → modifica estado →
`invoke(None)`

CANCELAR

Nao chamar `invoke(None)` — a thread fica
suspensa até expirar

O grafo pesquisador — loop visível no draw_mermaid()

A aresta de volta **avaliar** → **buscar** cria um ciclo — impossível em chains LCEL, explícito em LangGraph e visível no diagrama Mermaid:



O CICLO E O DIFERENCIAL — E O RISCO

A aresta **avaliar** → **buscar** cria um ciclo. Sem o limite MAX_ITER, o grafo poderia executar infinitamente. Sempre incluir uma condição de saída baseada em contagem de iterações — nunca confiar só na qualidade para parar o loop.

07

Lab — Notebook Aluno (25 min)

60% de lacunas — construir o grafo pesquisador para o domínio do grupo.
Mínimo: 3 nodes + 1 conditional edge + `draw_mermaid()` mostrando o loop.

```
!pip install langgraph langchain-ollama langchain-community duckduckgo-search -q
```

```
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from typing import TypedDict, Annotated
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_ollama import ChatOllama
from langchain_community.tools import DuckDuckGoSearchRun
from IPython.display import Image, display
import os
from google.colab import userdata
os.environ["OLLAMA_HOST"]="https://ollama.com"
os.environ["OLLAMA_API_KEY"]=userdata.get("OLLAMA_API_KEY")
llm=ChatOllama(model="qwen3.6:27b",temperature=0)
```

```
# LACUNA 1: Estado com TypedDict
class Estado(TypedDict):
    mensagens: Annotated[list, ( )] # usar add_messages
    resultados_busca: str
    qualidade: ( ) # float
    iteracoes: ( ) # int
```

```
class Qualidade(BaseModel):
    score:float; suficiente:bool
```

```
# LACUNA 2: node_buscar
def node_buscar(state:Estado)->dict:
    bruto=DuckDuckGoSearchRun().run(( )) # query = mensagens[0].content
    return {"resultados_busca":bruto[:1500],"iteracoes":state["iteracoes"]+1}

def node_avaliar(state:Estado)->dict: # PRONTO
    q=(ChatPromptTemplate.from_template("P:{p}
R:{r}
Score:")
    |llm.with_structured_output(Qualidade)).invoke(
        {"p":state["mensagens"][0].content,"r":state["resultados_busca"]})
    return {"qualidade":q.score}
```

```
# LACUNA 3: node_responder
def node_responder(state:Estado)->dict:
    resp=llm.invoke([HumanMessage(
        f"P:{state['mensagens'][0].content}
Fonte:{state[( )]}")
    ))
    return {"mensagens":[resp]}
```

```
# LACUNA 4: decidir_continuar (threshold 0.7, MAX 3)
def decidir_continuar(state:Estado)->str:
    if state["qualidade"]>=( ) or state["iteracoes"]>=( ):
        return ( ) # "responder"
    return ( ) # "buscar"
```

```
# LACUNA 5: montar grafo
builder=StateGraph(Estado)
builder.add_node("buscar",( ));builder.add_node("avaliar",( ));builder.add_node("responder",( ))
builder.add_edge(START,"buscar");builder.add_edge("buscar",( )) # → avaliar
builder.add_conditional_edges("avaliar",( ),{"buscar":"buscar","responder":"responder"})
builder.add_edge("responder",END)
pesquisador=builder.compile(checkpointer=MemorySaver())
try: display(Image(pesquisador.get_graph().draw_mermaid_png()))
except: print(pesquisador.get_graph().draw_mermaid())
```

Sintese da Aula 13

- ✓ **StateGraph = TypedDict + Nodes + Edges.** O estado e o dict compartilhado; nodes retornam apenas o que mudou; compile() valida e retorna Runnable.
 - ✓ **add_conditional_edges() torna a decisão explícita e visível.** A função de roteamento lê o estado; o dict mapeia o retorno para o próximo nó. draw_mermaid() mostra o grafo incluindo loops.
 - ✓ **MemorySaver habilita estado persistente por thread_id.** Múltiplas sessões paralelas sem interferência. get_state() inspeciona o estado salvo a qualquer momento.
 - ✓ **HITL com interrupt_before** pausa antes de ações irreversíveis. invoke(None) retoma; update_state() permite editar antes de continuar.
- **Aula 14 — Encerramento:** tendências 2026, LangGraph Platform, próximos passos na carreira e retrospectiva do semestre.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Por que Annotated[list, add_messages] em vez de list simples?"	Com list simples, cada update de node substitui a lista inteira, perdendo o histórico. Annotated[list, add_messages] usa um reducer que acumula: quando um node retorna {"mensagens": [nova_msg]}, o LangGraph faz merge, adicionando a lista existente. Campos sem reducer (int, str, float) são sempre substituídos pelo último update.
"MemorySaver em produção e seguro?"	Não — MemorySaver é in-memory e perde tudo ao reiniciar. Para produção usar SqliteSaver (arquivo local) ou PostgresSaver (banco de dados). A API é idêntica — só troca o checkpointer no compile(). Para CKP e demos, MemorySaver é suficiente.
"draw_mermaid_png() deu erro no Colab — como resolver?"	Requer playwright: <code>!pip install playwright && playwright install chromium</code> . Alternativa sem instalação: usar draw_mermaid() e colar o output em mermaid.live. Renderiza online instantaneamente.
"O grafo pode ter múltiplos interrupt_before?"	Sim — interrupt_before=["no_A", "no_B"] pausa antes de qualquer um dos dois. Também existe interrupt_after — pausa depois do no (o no executa, mas o próximo fica suspenso). Para pipelines com múltiplos checkpoints de aprovação, usar lista de nós.

Python novo desta aula

CONCEITOS_PYTHON_AULA13_2SEM.PY

```
# 1. TypedDict – dicionario com tipos declarados
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class Estado(TypedDict):
    mensagens: Annotated[list, add_messages] # reducer: acumula
    contador: int # sem reducer: substituido no update

# 2. app.get_state(config) – inspecionar estado salvo
snapshot = app.get_state({"configurable":{"thread_id":"abc"}})
snapshot.values # → dict com estado atual
snapshot.next # → () se terminou, ('nome_no',) se pausado

# 3. app.update_state(config, values) – editar estado antes de retomar
app.update_state(
    {"configurable":{"thread_id":"abc"}},
    {"mensagens":[HumanMessage("versao editada")]},
)

# 4. app.invoke(None, config) – retomar grafo pausado
app.invoke(None, config=config)
# None = continuar do ponto suspenso sem novo input

# 5. pesquisador.get_graph().draw_mermaid() – auto-documentacao
print(pesquisador.get_graph().draw_mermaid())
# retorna string Mermaid – colar em mermaid.live para visualizar
# ou usar draw_mermaid_png() para renderizar inline no Colab
```

LangGraph em producao — padroes de grafo

PLATAFORMAS DE RECRUTAMENTO

Agente de recrutamento

Agente com LangGraph e loop de avaliação de currículo: buscar candidatos → avaliar fit → se score baixo, reformular critérios e buscar de novo → se score alto, agendar entrevista (HITL antes do contato). O padrao desta aula em escala.

AGENTES DE CODIFICACAO AUTONOMA

Coding loop

Grafo com loop: gerar código → rodar testes → se falhou (conditional edge) → corrigir e rodar de novo → se passou → entregar. O ciclo roda até os testes passarem ou atingir o limite de tentativas.

CRMS COM IA

Grafo de vendas

Funil de vendas como grafo de estado: qualificar lead → proposta → negociacao → fechamento. Conditional edges decidem se avanca, volta ou escalona para humano. Estado persistido por oportunidade no CRM.

Mapa completo do 2º semestre

MODULO	AULAS	CONCEITO CENTRAL	CKP
M1 LangChain	01-04	LCEL, ChatOllama, Memoria, Pydantic v2, Context Engineering	CKP01 ✓
M2 RAG	05-07	Embeddings, ChromaDB, Pipeline RAG, Chunking, RAGAS	CKP02 ✓
M3 Agentes	08-11	Gradio, ReAct, AgentExecutor, Context rot, Integracao	CKP03 ✓
M4 LangGraph	12-14	Router Chain, StateGraph, Conditional edges, MemorySaver, HITL	—



Preview — Aula 14: Encerramento

CONTEUDO DA AULA 14

Tendencias 2026: LangGraph Platform, multi-agent, observabilidade com LangSmith

Proximos passos na carreira: MLOps para agentes, contribuicao open source, o que aprender depois

Retrospectiva: do chain de 5 linhas ao agente com HITL — a progressao completa

Aula leve e descartavel — pode ser pulada se o semestre atrasou

PARA ONDE O LANGGRAPH ESTA INDO



LangGraph Platform — deploy na nuvem com autoscaling e API REST automatica



Multi-agent — grafos onde cada no e um agente especializado, coordenados por supervisor



LangSmith — trace cada execucao do grafo, debugar conditional edges, medir latencia

06

Demo — Agente pesquisador com loop condicional

buscar → avaliar qualidade → decidir: buscar mais (loop) ou responder (END).
Maximo 3 iteracoes. O loop aparece no draw_mermaid().

Demo Prática ao Vivo — agente pesquisador com loop condicional

AGENTE_PESQUISADOR.PY

```
from langgraph.graph import StateGraph, START, END
from langchain_community.tools import DuckDuckGoSearchRun
from pydantic import BaseModel
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langchain_core.messages import HumanMessage
from langchain_core.prompts import ChatPromptTemplate

class EstadoPesquisa(TypedDict):
    mensagens: Annotated[list, add_messages]
    resultados_busca: str
    qualidade: float
    iteracoes: int

class Qualidade(BaseModel):
    score: float; suficiente: bool

def node_buscar(state) -> dict:
    bruto = DuckDuckGoSearchRun().run(state["mensagens"][0].content)
    return {"resultados_busca":bruto[:1500], "iteracoes":state["iteracoes"]+1}

def node_avaliar(state) -> dict:
    q = (ChatPromptTemplate.from_template(
        "Pergunta:{p}
Resultado:{r}
Score suficiencia 0.0-1.0:"
    ) | llm.with_structured_output(Qualidade)).invoke(
        {"p":state["mensagens"][0].content, "r":state["resultados_busca"]}
    )
    return {"qualidade": q.score}

def node_responder(state) -> dict:
    resp = llm.invoke([HumanMessage(
        f"Pergunta:{state['mensagens'][0].content}
Fonte:{state['resultados_busca']}"
    )])
    return {"mensagens":[resp]}

def decidir_continuar(state) -> str:
    # Para se qualidade OK ou atingiu limite de 3 iteracoes
    if state["qualidade"] >= 0.7 or state["iteracoes"] >= 3:
        return "responder"
    return "buscar" # loop de volta

builder = StateGraph(EstadoPesquisa)
builder.add_node("buscar",node_buscar); builder.add_node("avaliar",node_avaliar)
builder.add_node("responder",node_responder)
builder.add_edge(START,"buscar"); builder.add_edge("buscar","avaliar")
builder.add_conditional_edges("avaliar",decidir_continuar,
    {"buscar":"buscar","responder":"responder"})
builder.add_edge("responder",END)
pesquisador = builder.compile(checkpointer=MemorySaver())
```

Grafo pesquisador com loop condicional para o dominio ★★★

Grupo 3-4 · 25 minutos · Google Colab

1. **Complete as 5 lacunas** — TypedDict com add_messages + tipos; node_buscar com DuckDuckGo; node_responder usando resultados_busca; decidir_continuar com threshold 0.7 e MAX 3; montar o grafo com add_node, add_edge e add_conditional_edges.
2. **Visualize o grafo** com draw_mermaid() — confirmar que a aresta de volta (avaliar → buscar) aparece no diagrama.
3. **Execute 2 perguntas do dominio** e observe: quantas iteracoes foram necessárias? O threshold 0.7 foi atingido antes do limite 3 ou precisou do MAX?
4. **Experimento com threshold:** mudar para 0.5 e 0.9 e documentar a diferença no número de iteracoes em celula markdown.

Gabarito das lacunas

```
L1: add_messages ; float ; int
L2: state["mensagens"][0].content
L3: state["resultados_busca"]
L4: 0.7 ; 3 ; "responder" ; "buscar"
L5: node_buscar ; node_avaliar ; node_responder ; "avaliar" ; decidir_continuar
```

Desafio — HITL no pesquisador: adicionar `interrupt_before=["responder"]` para que o humano aprove antes da resposta final. Testar: o grafo pausa? `get_state()` mostra o no suspenso? `invoke(None)` retoma?

Referencias

DOCS **LangGraph** — Guia completo: StateGraph, checkpointing, HITL. langchain-ai.github.io/langgraph/tutorials/introduction

DOCS **LangGraph HITL** — interrupt_before, update_state, invoke(None). langchain-ai.github.io/langgraph/concepts/human_in_the_loop

BLOG **Anthropic Engineering** — "Building Effective Agents" (2025). Secao sobre checkpointing e revisao humana. anthropic.com/engineering/building-effective-agents

TOOL **Mermaid Live Editor** — Para visualizar o output de draw_mermaid() sem instalar playwright. mermaid.live

LIVRO **Russell, S.; Norvig, P.** — Inteligencia Artificial. 3ª ed. Pearson, 2016. Cap. 3 — Resolucao de problemas como busca: base conceitual dos grafos de estado no LangGraph.

Grafos em código

StateGraph, conditional edges, MemorySaver e HITL. O fluxo é agora explícito, auditável e pode pausar para o humano aprovar. Semestre quase encerrado.



PROXIMA AULA — AULA 14 (ULTIMA)

Spec-Driven Development e Encerramento

Retrospectiva do semestre · Spec-Driven Development · LangSmith ·
Proximos passos na carreira · Encerramento.